第 4 章　数组、广义表和字符串

4.1 引言

线性表、栈和队列都是一维的线性结构，其中的每个元素都是不可分割的原子类型。但在实际应用中，线性结构既会有维度上的拓展（例如二维数组以及更多维的数组形式）也会有深度上的拓展（例如广义表等形式），数据的元素既可以是原子类型也可以是嵌套的结构类型。

还有一种特殊的线性结构，那就是字符串。可以将字符串看成是由字符类型的元素组成的线性表，计算机的信息检索、文字编辑等都是以字符串数据作为处理对象。它与一般线性表的区别是，每次操作或处理的对象不仅是单个字符元素，还经常会是字符串的一部分或整体。

4.2 数组

我们在 C/C++程序设计中已经学习和使用过**数组**（array）。数组是存储于一个连续存储空间中的相同数据类型的数据元素的集合，通过数组元素的下标可以找到存放该数组元素的存储地址，从而可以访问该数组元素的值。就此意义来看，可以将数组看作线性表。

4.2.1 一维数组

一维数组是由具有相同数据类型的n（$n \geq 0$）个元素组成的有限序列，其中n为数组**长度**或数组大小，$n = 0$的数组是**空数组**。这些元素是存储在一组连续的存储空间中，因此可以将一维数组看成是由数组元素组成的、以顺序结构存储的线性表。实际上，线性表的顺序存储通常就是采用数组完成的。数组元素既可以是基本数据类型也可以是结构类型，但必须是同一数据类型。在数学上，也可以将一维数组看成为向量。

图 4.1 展示了一维数组存储结构。设数组首地址为b，数组元素长度为c，则第i个元素所在地址为$LOC(a_i) = b + i \times c$。

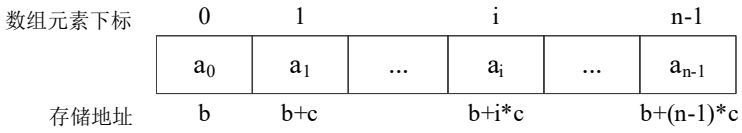


图 4.1 一维数组存储结构示意图

4.2.2 二维数组

二维数组在数学中可以看成一个矩阵，在二维数组$a[n][m]$中，共有$n \times m$个数据元素。也可以将二维数组看作由n个行向量或m个列向量所组成的矩阵，如图 4.2 所示。

每一元素$a[i][j]$同时处于第i个行向量和第j个列向量之中，如果存在，它在行的方向和列的方向各有一个直接前驱（$a[i-1][j]$和$a[i][j-1]$）以及一个直接后继（$a[i+1][j]$和$a[i][j+1]$）。因此，每一个数组元素在数组中的位置由行、列下标的二元组唯一确定。

若将数组中每个行向量看成一个元素，那么二维数组$a[n][m]$可以看成由n个行向量作为元素的线性表；同理，若将数组中每个列向量看成一个元素，那么二维数组$a[n][m]$可以看成由m个列向量作为元素的线性表。

$$\begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0(m-1)} \\ a_{10} & a_{11} & \cdots & a_{1(m-1)} \\ \vdots & \vdots & \vdots & \vdots \\ a_{(n-1)0} & a_{(n-1)1} & \cdots & a_{(n-1)(m-1)} \end{bmatrix}$$

图 4.2 二维数组结构示意图

二维数组的存储也采用顺序结构，共有两种方式，即行优先顺序和列优先顺序。以图 4.2 为例，按照行优先的顺序，所有数组元素按行向量依次排列，第$i+1$个行向量紧跟在第i个行向量后面，这样得到数组元素存储的一种按行序的线性序列：

$a[0][0], a[0][1], \ldots, a[0][m-1], a[1][0], a[1][1], \ldots, a[1][m-1], \ldots, a[n-1][0], a[n-1][1], \ldots, a[n-1][m-1]$

大多数程序设计语言（如 PASCAL、C/C++、BASIC 等）都是按行优先的顺序把数组元素存于内存中。下面我们就行优先的顺序讨论地址的映射方法。

设二维数组$a[n][m]$的第一个元素$a[0][0]$在内存中的地址为$LOC(0,0)$，每个元素占c个存储空间，那么应如何求任一数组元素$a[i][j]$在内存中的地址$LOC(i,j)$？

这里先考虑数组元素在相应的存储序列中的序号。设$a[0][0]$序号为 1，$a[0][1]$序号为 2。在二维数组$a[n][m]$中，$a[i][j]$所处位置上面已有i行，每行m 个元素，因此共有$i \times m$个元素，而$a[i][j]$在所处行上位于第$j+1$位，所以$a[i][j]$的序号为$k = i \times m + j + 1$。若序号为 1 的元素地址为$LOC(1)$，序号为k的元素地址为$LOC(k)$，元素大小为c，显然有$LOC(k) = LOC(1) + (k-1) \times c$。所以，数组元素$a[i][j]$在内存中的地址为

$$LOC(i,j) = LOC(0,0) + (i \times m + j) \times c \qquad (4.1)$$

二维以上的数组称为多维数组，它在内存中也存储在一片连续的存储空间中，因此其元素必须排列成一种线性序列。就像二维数组的元素可以按行优先的顺序排列一样，三维数组也可以按页优先的顺序排列。设三维数组$a[m_1][m_2][m_3]$，排列过程先考虑第三维变化，然后是第二维变化，最后是第一维变化。对于数组$a[i][j][k]$，i 每增加 1，就增加一页，也就是$m_2 \times m_3$个元素；j 每增加 1，就增加m_3个元素。由此可推出$a[i][j][k]$的序号为$i \times m_2 \times m_3 + j \times m_3 + k + 1$，因此$a[i][j][k]$在内存中的地址为

$$LOC(i,j,k) = LOC(0,0,0) + (i \times m_2 \times m_3 + j \times m_3 + k) \times c \qquad (4.2)$$

同理可推广到更高维的数组。

4.3 特殊矩阵的压缩存储

矩阵是很多科学与工程计算中研究的数学对象，然而在数值分析中经常出现一些维数很高的矩阵，在这些矩阵中有许多值相同的元素或零元素，例如数字图像处理中的图像数据。为了节省存储空间，可对这类矩阵进行压缩，即对多个值相同的元素只分配一个存储空间，对零元素不分配存储空间。若值相同的元素或零元素在矩阵中分布有一定规律，则称此类矩阵为特殊矩阵 (special matrix)。

4.3.1 对称矩阵

若n 阶矩阵a中的元素满足条件$a_{ij} = a_{ji}$（$0 \le i, j \le n-1$），则称a为n阶对称矩阵。

对于对称矩阵，可考虑为每一对对称的相同元素分配一个存储空间，这样就可以将原来需要的n^2个存储空间压缩存储到$n(n+1)/2$个存储空间。不失一般性，可以按照行优先的顺序存储对称矩阵的下三角(包括对角线)元素。

设$a[0][0]$的存储地址为$LOC(0,0)$，每个元素所占的存储空间为c，为求$a[i][j]$的存储地址$LOC(i,j)$，我们同样采用首先计算元素在存储序列中的序号k的方法。

设$a[0][0]$序号为 1，在下三角矩阵（如图 4.3a 所示）中，元素$a[i][j]$（$i \geq j$）上面有i行，第 1 行有 1 个元素，第 2 行有 2 个元素，以此递增，则该元素上面共有$i(i+1)/2$个元素，而元素$a[i][j]$在所在行上处于第$j+1$位，因此其序号$k = i(i+1)/2 + j + 1$。由此可得，数组元素$a[i][j]$在内存中的地址为

$$LOC(i,j) = LOC(0,0) + (k-1) \times c = LOC(0,0) + (i(i+1)/2 + j) \times c \quad (4.3)$$

下三角矩阵的其余元素可通过$a_{ij} = a_{ji}$求得。这种压缩存储方法同样适用于上三角矩阵。

同理可得上三角矩阵的计算公式。在上三角矩阵（如图 4.3b 所示）中，元素$a[i][j]$（$i \leq j$）上面有i行，第 1 行有n个元素，第 2 行有$n-1$个元素，以此递减，则该元素上面共有$i(n+n-i+1)/2$个元素，而元素$a[i][j]$在所在行上处于第$j-i+1$位，因此其序号$k = i(2n-i+1)/2 + j - i + 1$。由此可得，数组元素$a[i][j]$在内存中的地址为

$$LOC(i,j) = LOC(0,0) + (k-1) \times c = LOC(0,0) + (i(2n-i+1)/2 + j - i) \times c \quad (4.4)$$

$$\begin{bmatrix} a_{00} & & & \\ a_{10} & a_{11} & & \\ \vdots & \vdots & a_{ii} & \\ a_{(n-1)0} & a_{(n-1)1} & \cdots & a_{(n-1)(n-1)} \end{bmatrix} \qquad \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0(n-1)} \\ & a_{11} & \cdots & a_{1(n-1)} \\ & & a_{ii} & \cdots \\ & & & a_{(n-1)(n-1)} \end{bmatrix}$$

a）下三角矩阵　　b）上三角矩阵

图 4.3 三角矩阵

4.3.2 对角矩阵

在数值分析中还经常出现一种称为对角矩阵的特殊矩阵，这种矩阵的非零元素都集中在以对角线为中心的带状区域，图 4.4 所示的是带宽为 3 的三对角矩阵。

$$\begin{bmatrix} a_{00} & a_{01} & & & & 0 \\ a_{10} & a_{11} & a_{12} & & & \\ & \ddots & \ddots & \ddots & & \\ & & & & & a_{(n-2)(n-1)} \\ 0 & & & a_{(n-1)(n-2)} & & a_{(n-1)(n-1)} \end{bmatrix}$$

图 4.4 三对角矩阵

设$a[0][0]$序号为 1，三对角矩阵中（如图 4.4 所示），元素$a[i][j]$（$|i-j| \leq 1$）上面有i行，第 1 行有 2 个元素，第 2 行以下每行均有 3 个元素，则上面共有$3 \times (i-1) + 2$个元素（$i > 0$），元素在所在行上处于第$j-i+2$位，因此其序号$k = 3 \times (i-1) + 2 + j - i + 2$。由此可得，该数组元素在内存中的地址为

$$LOC(i,j) = LOC(0,0) + (k-1) \times c = LOC(0,0) + (3 \times (i-1) + 2 + j - i + 1) \times c \quad (4.5)$$

在实际应用中，还经常遇到另一类矩阵，其非零元素较少且分布没有规律，这就是下一节要讨论的稀疏矩阵。

4.4 稀疏矩阵的压缩存储

若矩阵中的非零元素非常少（但无明确规定），我们称这类矩阵为稀疏矩阵(sparse matrix)。一般地，对于一个$m \times n$的矩阵， $s = m \times n$为矩阵元素总数。设t为矩阵非零元素的个数，若$t \ll s$，则可认为该矩阵为稀疏矩阵。

4.4.1 稀疏矩阵的三元组表示

在实际应用中，需要处理的稀疏矩阵常常是很大的。例如，一个只有 99 个非零元素的 1000×1000 矩阵。显然，要存储矩阵中所有的 1000000 个元素是不合适的，只需存储其中的 99 个非零元素，因此需要设计稀疏矩阵的压缩存储方式。

由于非零元素的分布没有规律，所以在矩阵中可用一个三元组（行，列，值）确定一个矩阵的非零元素。由此可将非零元素的三元组作为线性表的元素来表示一个稀疏矩阵，如图 4.5 所示。

$$\begin{bmatrix} 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 6 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 7 & 0 & 0 & 0 \end{bmatrix}_{5\times6}$$

（0, 2, 4）
（1, 1, 6）
（3, 0, 5）
（3, 4, 3）
（4, 2, 7）

图 4.5 稀疏矩阵（左）及其三元组表示（右）

由于矩阵运算的种类很多，在下列抽象数据类型稀疏矩阵的定义中，我们只列举了几种常见的运算，抽象数据类型稀疏矩阵的定义如下。

```
ADT SparseMatrix
{
    数据对象：D={a_ij | i = 1,2,...,n； j = 1,2,...,m； a_ij ∈ ElemSet，n和m分别称为矩阵的行数和列数}
    数据关系： R={Row，Co1}
              Row={< a_ij, a_i(j+1) > | 1 ≤ i ≤ n, 1 ≤ j ≤ m − 1 }
              Col={< a_ij, a_(i+1)j > | 1 ≤ i ≤ n − 1, 1 ≤ j ≤ m }
    基本操作：
        // 1. 初始化和销毁操作
        CreateSMatrix( &M )
            初始条件：无
            操作结果：创建稀疏矩阵 M
        DestroySMatrix( &M )
            初始条件：稀疏矩阵 M 存在
            操作结果：销毁稀疏矩阵 M
        // 2. 访问型操作
        PrintSMatrix( M )
            初始条件：稀疏矩阵 M 存在
            操作结果：输出稀疏矩阵 M
        // 3. 加工型操作
        CopySMatrix( M, &T )
            初始条件：稀疏矩阵 M 存在
            操作结果：由稀疏矩阵 M 复制得到 T
        AddSMatrix( M,  N, &Q )
            初始条件：稀疏矩阵 M 与 N 的行数和列数对应相等
            操作结果：求稀疏矩阵的和 Q=M+N
        SubSMatrix( M,  N, &Q)
```

初始条件：稀疏矩阵 M 与 N 的行数和列数对应相等

操作结果：求稀疏矩阵的差 Q=M-N

MultSMatrix(M, N, &Q)

初始条件：稀疏矩阵 M 的列数等于 N 的行数

操作结果：求稀疏矩阵乘积 Q=M×N

TransposeSMatrix(M, &T)

初始条件：稀疏矩阵 M 存在

操作结果：求稀疏矩阵 M 的转置矩阵 T

} // ADT SparseMatrix

实现三元组表的存储，即稀疏矩阵的压缩存储，既可以利用顺序结构存储，也可利用链式结构存储，这就是下面介绍的三元组顺序表表示和三元组十字链表表示。

4.4.2 三元组的顺序表表示

在三元组的顺序表中，矩阵的非零元素的三元组按照在原矩阵中的位置以行优先的顺序依次存放，组成一个以三元组为元素的顺序表，此外还要存储原矩阵的行数、列数和非零元素个数。基于以上要求，下面给出稀疏矩阵的顺序表表示的定义。

```
#define MAXSIZE 256  // 设定非零元素个数初始最大值
typedef int ElemType;   // 假设元素类型为整型
typedef struct
{
    int         i, j;          // 数据所在的行、列值
    ElemType v;                // 数据元素值
} Triple;                      // 三元组定义
typedef struct
{
    Triple arr[MAXSIZE]; // 非零元素三元组
    int Rows,Cols,Nums;   // 矩阵的行数、列数、非零元素个数
}SqSMatrix;                    //  稀疏矩阵的三元组顺序表示
```

基于该定义，下面讨论如何进行稀疏矩阵的转置。

例 4-1 稀疏矩阵的转置 稀疏矩阵的转置运算是一种最简单的矩阵运算。图 4.6c 中的矩阵 B 是矩阵 A（图 4.6a）的转置矩阵，这两个矩阵的相应三元组表分别如图 4.6b 和图 4.6c 所示。

$$\begin{bmatrix} 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 6 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 7 & 0 & 0 & 0 \end{bmatrix}_{5\times6}$$

a) 矩阵 A

行	列	值
0	2	4
1	1	6
3	0	5
3	4	3
4	2	7

b) A.arr

$$\begin{bmatrix} 0 & 0 & 0 & 5 & 0 \\ 0 & 6 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 7 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}_{6\times5}$$

c) 转置矩阵 B

行	列	值
0	3	5
1	1	6
2	0	4
2	4	7
4	3	3

d) B.arr

图 4.6 用三元组表示的稀疏矩阵及其转置

从图中的 A.arr 三元组表计算 B.arr 三元组表一般有三种常用的方法。

1. 简单方法

把 A.arr 三元组表中的行与列的内容互换，然后再按行号对新的三元组表中的各三元组从小到大进行稳定排序，就可得到 B.arr 的三元组表，其操作为经典的排序算法，时间复杂度为 $O(A.Nums^2)$ 。

2. 列序遍历方法

其算法思想是按照 B.arr 中三元组的次序，在 A.arr 中找到相应的三元组进行转置。根据 B.arr 中三元组的行序，首先在 A.arr 中找列号为 0 的三元组，将其转置并使之成为 B.arr 中行号为 0 的三元组，以此类推。假设稀疏矩阵 A 有 n 列，则相应地需要针对它的三元组表中的列项进行 n 趟扫描，第 k 趟扫描是在 A.arr 的 j 项中查找列号为 k 的三元组，若存在，则意味着这个三元组所表示的元素在稀疏矩阵 A 的第 k 列，需要把它存放到转置矩阵 B 的第 k 行。具体实现方法是：取出这个三元组，并交换其 i（行号）与 j（列号）的内容，连同 v 的值作为新三元组存放到转置矩阵的三元组表中，当 n 趟扫描完成时，转置完成。算法具体实现如下。

算法 4.1 列序遍历的转置方法

```
Status TransposeSMatrix( SqSMatrix A, SqSMatrix &B )
{
    // 将稀疏矩阵 A 转置，结果在稀疏矩阵 B 中
    B.Rows  = A.Cols;  // 矩阵 B 的行数等于矩阵 A 的列数
    B.Cols   = A.Rows; // 矩阵 B 的列数等于矩阵 A 的行数
    B.Nums = A.Nums;  // 矩阵 B 的非零元素数等于矩阵 A 的非零元素数
    if ( A.Nums > 0 )
    {   // 若非零元素个数不为零
        q = 0;  // B.arr 的当前位置
        for( k=0; k<A.Cols;  k++)  // 按列号对 A 做 Cols 趟扫描
        {
            for( p=0; p<A.Nums; p++ ) // 每趟对 A 所有三元组进行扫描
            {
                if ( A.arr[p].j == k )
                {   // 若第 p 个三元组中元素的列号为 k
                    B.arr[q].i = A.arr[p].j;   // 新三元组中的行号
                    B.arr[q].j = A.arr[p].i;   // 新三元组中的列号
                    B.arr[q].v = A.arr[p].v; // 新三元组中的值
                    q++;   // B.arr 的当前位置增加 1
                } //end if  (A.arr[p].j==k)
            } //end for p
        } // end for k
    } //end if  ( A.Nums>0 )
    return OK;
} // TransposeSMatrix
```

不难看出，以上算法的时间复杂度为$O(A.Cols \times A.Nums)$。显然，当矩阵 A 的列数较小时，该算法有一定的优势，当 A 的列数较大时，该算法的时间复杂度也会很高。

3. 快速转置法

沿着第二种方法的思路，思考能否在转置时，不根据列号每次都遍历，而是只遍历整个三元组两次，就能直接将各三元组直接放到转置后所在的新位置。显然，可以在第一次遍历稀疏矩阵 A 的三元组时，统计出 A 中每列的三元组个数，即转置后稀疏矩阵 B 中每行的三元组个数。这样，就可以方便地得到转置后稀疏矩阵 B 中每行三元组的起始位置。在第二次遍历稀疏矩阵 A 的三元组时，每访问到一个三元组，就可以直接将其放到稀疏矩阵 B 中相应的位置。

为了能快速确定 A 中每个三元组在 B 中的位置，这里我们引入两个辅助数组。

（1） rowNum[col]：存放稀疏矩阵 A 各列的非零元素个数，也就是转置矩阵 B 各行的非零元素个数。具体做法是：先把这个数组清零，然后遍历矩阵 A 的三元组表，逐个取出三元组的列号 col，把以此列号为下标的辅助数组元素的值累加 1。

（2） rowStart[col]：存放转置矩阵 B 中各行三元组应存放的起始位置。显然有

rowStart[col] = 0 （col=0)

rowStart[col] = rowStart[col-1] + rowNum[col-1] （1≤col≤Cols）

例如，对图 4.6a 所示的稀疏矩阵 A，事先计算的 rowNum 和 rowStart 如下表 4.1 所示。

表 4.1 稀疏矩阵快速转置的辅助数组

col	0	1	2	3	4	5
rowNum	1	1	2	0	1	0
rowStart	0	1	2	4	4	5

具体来说，快速转置算法思想如下。

第 1 步：对稀疏矩阵 A 进行第一趟遍历，根据矩阵 A 的列号，统计 A 中各列非零元素的个数，即转置后各行非零元素个数。

第 2 步：根据第 1 步的统计结果，计算在转置矩阵 B 的三元组表中各行对应的起始位置。

第 3 步：对稀疏矩阵 A 的三元组表进行第二趟遍历，对每个三元组交换其行号和列号，连同其值构成一个新的转置后的三元组。根据行号，按辅助数组所指示的位置，将该三元组直接存放到转置矩阵 B 中对应的位置上，并将该行的位置指示值加 1，为放置该行下一个三元组做准备。

其算法实现如下：

算法 4.2 稀疏矩阵的快速转置方法

```
Status FastTransposeSMatrix( SqSMatrix A, SqSMatrix &B )
{   // 将稀疏矩阵 A 转置，结果在稀疏矩阵 B 中。
    B.Rows = A.Cols;   // 矩阵 B 的行数=矩阵 A 的列数
    B.Cols  = A.Rows;  // 矩阵 B 的列数=矩阵 A 的行数
    B.Nums = A.Nums;  // 矩阵 B 的非零元素数=矩阵 A 的非零元素数
    if (A.Nums>0 )       // 矩阵非零元素个数不为零
    {  // 第 1 步：统计 A 中每列非零元素个数
       for ( k=0;  k<A.Cols;  k++ )  rowNum[k]=0;              // rowNum 数组初始化清零
       for ( p=0; p<A.Nums; p++ )  rowNum[A.arr[p].j]++;  // 统计 A 中每列（即 B 中每行）非零元素个数
       // 第 2 步：计算 B 中每行三元组的起始位置
       rowStart[0]=0;
       for ( k=1; k<A.Cols; k++ )
            rowStart[k] = rowStart[k-1] + rowNum[k-1];        // 矩阵 B 每行三元组的起始位置
       // 第 3 步：遍历 A 的三元组，进行矩阵转置
       for ( p=0; p<A.Nums;  p++)
       {
            q = rowStart[A.arr[p].j];  // 取出 B 当前行的起始位置
            B.arr[q].i = A.arr[p].j;     // 新三元组中的行号
            B.arr[q].j = A.arr[p].i;     // 新三元组中的列号
            B.arr[q].v = A.arr[p].v;    // 新三元组中的值
            rowStart[ A.arr[p].j ]++;  // B 当前行的起始位置增加 1，为该行下一个三元组的放入做准备
       } //end for
    } // end if ( A.Nums>0 )
    return OK;
} // FastTransposeSMatrix
```

显而易见，以上算法的时间复杂度为$O(A.Cols + A.Nums)$。

4.4.3 三元组的十字链表表示

当稀疏矩阵的非零元素位置和个数在操作过程中变化较大时，就不适合采用顺序表示的方式存储三元组。例如对于矩阵相加，如果采用顺序表示的方式存储三元组，则非零元素的插入和删除会引起顺序表中元素的大量移动，因此采用链式结构的三元组表示更为适合。

由于每个三元组包含了矩阵非零元素的行、列信息，所以链式结构通常不是采用简单的单链表形式，而是采用包含行、列指针的十字链表，其中每个结点不仅存储非零元素的三元组，还包含两个指针。也就是说，每个结点有五个域：i，j， v 三个域表示三元组；right 域为指向同一行下一非零元素的指针；down 域为指向同一列下一非零元素的指针。同一行的三元组通过 right 指针形成一个单链表，同一列的三元组通过 down 指针形成一个单链表，整个稀疏矩阵构成一个十字交叉的链表，故称为**十字链表**。

例如，图 4.6a 中的稀疏矩阵 A 的十字链表示如图 4.7 所示。

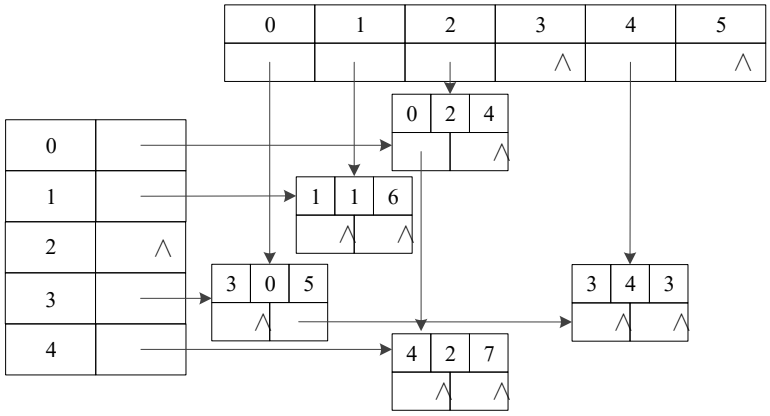


图 4.7 稀疏矩阵 A 的十字链表表示

4.5 广义表

顾名思义，广义表（generalized list）是线性表的推广。在人工智能领域使用的表处理语言 LISP 就是以广义表作为基本的数据结构，用其编写的程序也表示为一系列的广义表。

4.5.1 广义表的概念

广义表一般记为：$GL=(a_1,a_2,\dots,a_n)\ \ n\geq 0$。

其中GL是广义表$(a_1,a_2,\dots,a_n)$的名称，n是广义表的长度。在线性表中，a_i是单元素。在广义表中，a_i既可以是单元素，也可以是广义表，它们分别称为广义表的原子和子表。显然，广义表的定义是递归的。

一般用大写字母表示广义表和子表，用小写字母表示原子。当广义表GL非空时，第一个元素a_1为GL的表头（head），其余元素组成的子表$(a_2,\dots,a_n)$称为GL的表尾（tail）。例如：

(1) A=() A 是一个空表，其长度为 0。

(2) B=(a) B 的长度为 1，元素为原子，已退化为一般的线性表。

(3) C=(e，(a，b，c)) C 的长度为 2，第一个元素为原子，第二个元素为子表。

(4) D=(A，B，C) D 的长度为 3，三个元素都是子表，将三个子表代表的广义表代入后，可得 D=((),(a),(e,(a,b,c)))。

(5) E=(a,E) 这是一个递归的广义表，它长度为 2，是一个无限深度的广义表 E=(a,(a,(a,…)))。

从定义和例子中可得出下列结论：

(1) 广义表的元素既可以是原子，也可以是子表（广义表），子表的元素还可以是子表，因此广义表是有层次结构的。

(2) 广义表可被其它广义表所共享，如 A，B，C 为广义表，且都是 D 的子表。

(3) 广义表可以是一个递归的表。

如果用○表示原子，用□表示广义表，则可以用图形形象地表示广义表，如图 4.8 所示，它清楚地表示出广义表的层次结构。

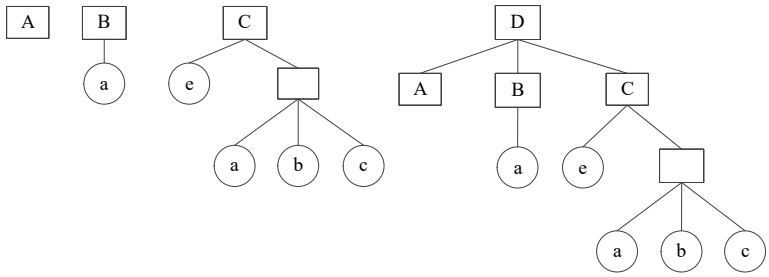


图 4.8 广义表的图形表示

取表头（GetHead）、取表尾（GetTail）是广义表的主要操作。根据上述对广义表的表头、表尾的定义可知：任何一个非空广义表，其表头既可能是原子，也可能是子表，其表尾一定是广义表。例如：

GetHead(B) = a
GetTail(B) = ()
GetHead(D) = A
GetTail(D) = (B, C)
GetHead(GetTail(D) = B
GetTail(GetTail(D)) = (C)

4.5.2 广义表的抽象数据类型

广义表的抽象数据类型定义如下。

```
ADT GList
{
    数据对象：D={e_i | i = 1,2, ..., n； n ≥ 0； e_i ∈ AtomSet或e_i ∈ GList， AtomSet为某个数据对象}
    数据关系：R={< e_{i-1}, e_i >| e_{i-1}, e_i ∈ D, i = 2, ..., n }
    基本操作：
        // 1. 初始化和销毁操作
        InitGList( &L )
            初始条件：无
            操作结果：创建空的广义表 L
        CreateGList( &L, S )
            初始条件：S 是广义表的书写形式串
            操作结果：由 S 创建广义表 L
        DestroyGList( &L )
            初始条件：广义表 L 存在
            操作结果：销毁广义表 L
        // 2. 访问型操作
        GListLength( L )
            初始条件：广义表 L 存在
            操作结果：求广义表 L 的长度，即第一层的元素个数
        GListDepth( L )
            初始条件：广义表 L 存在
            操作结果：求广义表 L 的深度
        GListEmpty( L )
            初始条件：广义表 L 存在
```

操作结果：判定广义表 L 是否为空，是空返回 TRUE，否则返回 FALSE

GetHead (L)

初始条件：广义表 L 存在

操作结果：取广义表 L 的表头

GetTail(L)

初始条件：广义表 L 存在

操作结果：取广义表 L 的表尾

TraverseGList(L)

初始条件：广义表 L 存在

操作结果：从广义表 第一个元素开始，依次访问并输出数据元素

// 3. 加工型操作

CopyGList(&T, L)

初始条件：广义表 L 存在

操作结果：由广义表 L 复制得到广义表 T

InsertFirstGList(&L, e)

初始条件：广义表 L 存在

操作结果：插入元素 e 作为广义表 L 的第一元素

DeleteFirstGList(&L, &e)

初始条件：广义表 L 存在

操作结果：删除广义表 L 的第一元素，并用 e 返回其值。

} // ADT GList

4.5.3 广义表的存储结构

由于广义表中的元素可以是原子，也可以是广义表，两者结构不同，大小不一，且难以用顺序结构存储，因此通常采用链式结构表示广义表。根据不同的信息存储方式，广义表的链式存储通常可采取两种不同的方式。无论采取什么方式，结点必须设计成两种类型：表结点和原子结点。

1. 层次结构

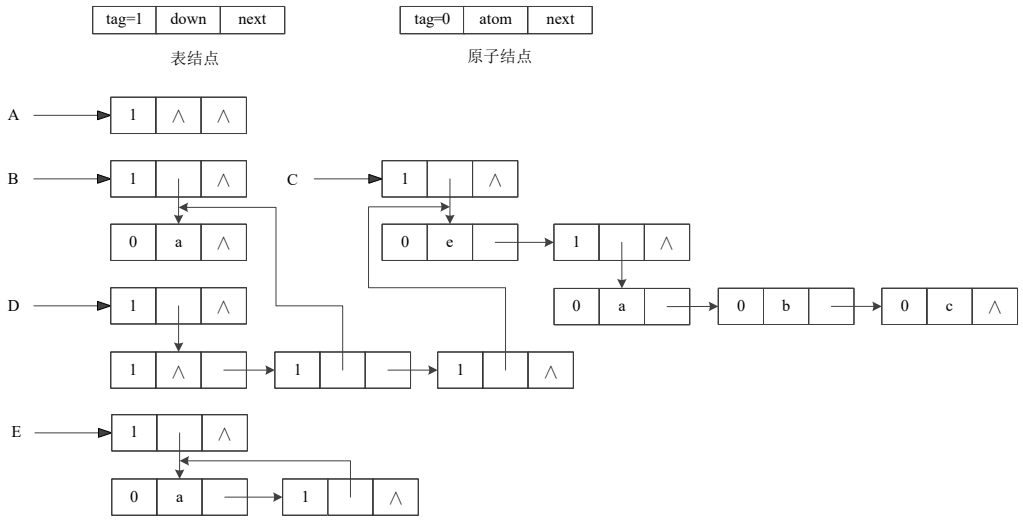


图 4.9 广义表的层次结构存储

层次结构考虑存储广义表的层次信息。在层次结构的链式存储中，表结点设有两个指针域(next，down)。next 指向同一层的下一个元素，down 指向下一层子表的第一个元素；原子结点不存在下一层次，因此只有一个 next 指针域指向同一层的下一个元素。两者用标志位 1 和 0 区别。如图 4.9 所示，这种存储方式具有明显的层次结构，因此适用于有层次要求的操作。

2. 表头表尾结构

表头表尾结构考虑存储广义表的表头表尾信息。在表头表尾结构中，表结点设有两个指针域(head，tail)，head 指向广义表的表头元素，tail 指向广义表的表尾；原子结点不设指针域，只存储元素值。两者用标志位 1 和 0 区别。如图 4.10 所示，在这种存储方式中，方便对广义表的表头表尾操作。

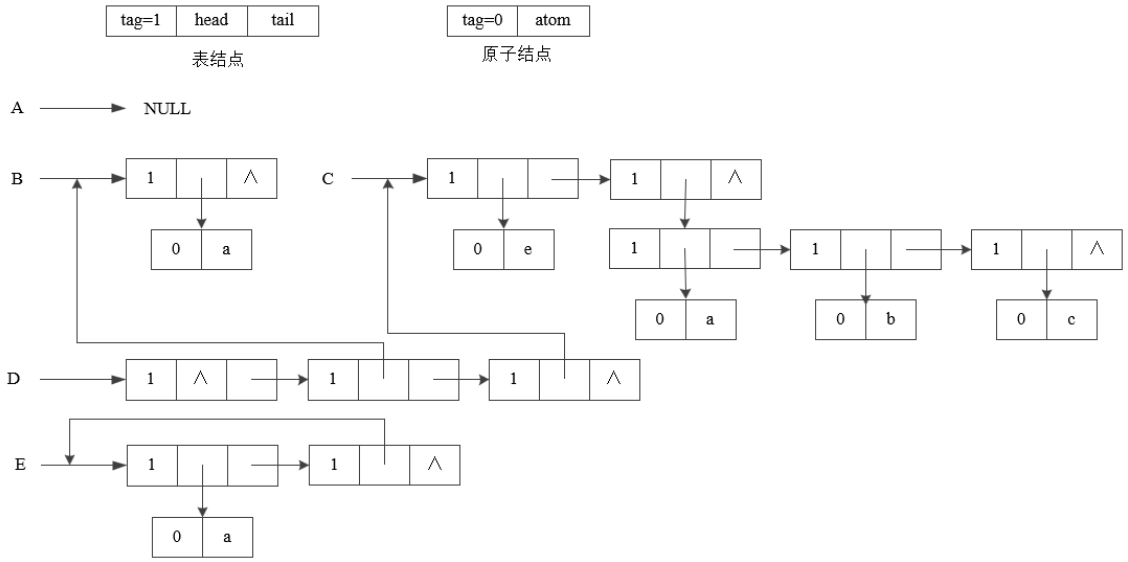


图 4.10 广义表的表头表尾结构存储

4.6 字符串

字符串在计算机信息处理中有广泛的应用，例如在计算机中使用最多的 Microsoft Word 软件，其主要功能----字符编辑就是以字符串结构处理的。

4.6.1 字符串的抽象数据类型

字符串（string）是由n（$n \geq 0$）个字符组成的有限序列。通常可记为

$$S = "a_1 a_2 \ldots a_n"（n \geq 0）\qquad（4.6）$$

其中，S是**字符串名**，既可以是字符串变量名，也可以是字符串常量名。双引号中的字符序列是字符串的**值**，字符串中的字符a_i可以是字母、数字或其他字符。n是字符串中的字符个数，也叫做字符串的**长度**，它不包括字符串结束符‘\0’。长度为零的字符串叫做**空串**，除字符串结束符外，它不包含任何其他字符。注意，空格是一个字符，只包含空格的字符串不是空串。从字符串中连续取出若干个字符组成的字符串称为**子串**，包含子串的字符串称为**主串**。字符在字符串序列的序号通常称为该字符在字符串中的**位置**，子串在主串中的位置以子串的第一个字符在主串中的位置来表示。

例如，有三个字符串 A="JING"、 B="NANJING"、 C="NAN JING"，它们的长度分别为 4、7、8。其中 A 是 B 和 C 的子串，它在 B 中的位置为 4，在 C 中的位置为 5。

可以对两个字符串进行比较操作。若两个字符串长度相等且对应位置的字符也相等，则称两个字符串相等，否则不相等。可以以字典排序的方法确定两个字符串的大小。

字符串虽然可以看成是一个由字符组成的线性表，但与一般的线性表有很大的区别。在一般线性表的基本操作中，大多以“单个元素”作为操作对象，例如在线性表中查找某个元素、求取

某个元素、在某个位置上插入一个元素和删除一个元素等；而在字符串的基本操作中，通常以“串的整体”作为操作对象，例如在字符串中查找某个子串、求取一个子串、在串的某个位置上插入一个子串以及删除一个子串等。这在下面的字符串的抽象数据类型定义可以看出。

字符串的抽象数据类型定义如下：

ADT String

{

数据对象：D = $\{a_i | a_i \in CharacterSet，\ i = 1,2,\dots,n，\ n \geq 0\}$

数据关系：R = { $< a_{i-1}, a_i > | a_{i-1}, a_i \in D，\ i = 2,\dots,n$ }

基本操作：

// **1. 初始化、清空和销毁操作**

StrAssign(&T, chars)

初始条件： chars 是字符串常量

操作结果：生成一个值为 chars 的字符串 T

ClearString(&S)

初始条件：字符串 S 存在

操作结果：将 S 清空为空串

DestroyString(&S)

初始条件：字符串 S 存在。

操作结果：字符串 S 被销毁。

// **2. 访问型操作**

StrEmpty(S)

初始条件：字符串 S 存在

操作结果：若 S 为空串，则返回 TRUE，否则返回 FALSE

StrCompare(S, T)

初始条件：字符串 S 和 T 存在

操作结果：若 S>T，则返回值大于 0；若 S=T，则返回值等于 0；若 S<T，则返回值小于 0

StrLength(S)

初始条件：字符串 S 存在。

操作结果：返回 S 的元素个数，即为字符串的长度

Substring(&Sub, S, pos, len)

初始条件：字符串 S 存在，1≤pos≤ StrLength(S)且 0≤1en≤StrLength(s)-pos+1

操作结果：用 Sub 返回字符串 S 的第 pos 个字符起长度为 1en 的子串

Index(S, T, pos)

初始条件：字符串 S 和 T 存在且都是非空串,1≤pos≤ StrLength(S)

操作结果：若主串 S 中存在和串 T 值相同的子串，则返回它在主串 S 中第 pos 个字符之后第一次出现的位置；否则返回 0

// **3. 加工型操作**

StrCopy(&T, S)

初始条件：字符串 S 存在

操作结果：由字符串 S 复制得字符串 T

Concat(&T, S1, S2)

初始条件：串 S1 和 S2 存在

操作结果：T 由 S1 和 S2 连接而成的新串，返回 T。

Replace(&S, T, V)

初始条件：字符串 S、T、V 存在，且 T 是非空串。

操作结果：用 V 替换主串 S 中出现的所有与 T 相等的不重叠的子串

StrInsert(&S, pos, T)

初始条件：字符串 S 和 T 存在,1≤pos≤ StrLength(S)+1

操作结果：在字符串 S 的第 pos 个字符之前插入字符串 T

StrDelete(&S, pos, len)

初始条件：字符串 S 存在, 1≤pos≤ StrLength(S)-1en+1
操作结果：从字符串 S 中删除第 pos 个字符起，长度为 1en 的子串
} // ADT String

字符串的基本操作可以有不同的定义方法，高级程序设计语言中的字符串操作也给出了字符串的基本操作。在上述抽象数据类型定义的 13 种操作中，字符串赋值 Str Assign、字符串比较 StrCompare、求字符串长 Strlength、串联接 Concat、求子串 SubString 等 5 种操作构成了串类型的最小操作子集。

4.6.2 字符串的存储结构与子串定位

由于字符串操作的特点，字符串在实际应用中基本上都是采用顺序存储结构，因为这种结构适合对整个字符串或子串进行操作。字符串的顺序存储结构是用一组连续的存储单元存放字符串的字符序列，它们的存储空间可以在程序中静态分配（如 C 语言中的字符型数组），也可以动态分配（如 C 语言中的字符串类型）。下面以子串定位 Index(S, T, pos)为例，说明顺序存储字符串的基本操作。

算法 4.3 子串定位算法

```
int Index( string S, string T, int pos)
{   // 若 T 是 S 子串，则返回其在 S 的 pos 个字符之后第一次出现的位置；否则返回 0
    k=pos-1; i=k; j=0;
    while( S[i]!='\0'&& T[j]!='\0' )
    {
        if ( S[i]== T[j] ) { i++; j++; } // 字符相等，继续匹配下一字符
        else { k++; i=k;  j=0; }          // 字符不等，从主串下一位置开始匹配
    }
    if ( T[j]=='\0' ) return k+1;          // 匹配成功
    else return 0;                         // 匹配不成功
} // Index
```

4.7 小结

本章讨论了数组、广义表和字符串，包括其抽象数据类型的描述、相应的存储表示与实现，以及相关的应用举例。

数组部分重点讨论了特殊矩阵的压缩存储和一般稀疏矩阵的三元组表示方法。特殊矩阵，例如对称矩阵、三角矩阵、对角矩阵等，可通过建立矩阵的行、列值与一维数组下标的映射关系，将原来的二维数组存储变换为一维数组的压缩存储。一般稀疏矩阵的三元组表示，可结合不同的应用场景，选择采用顺序存储或者链式存储，相应算法的设计需要考虑尽可能减少三元组的遍历次数。

广义表部分重点介绍了线性结构在深度层次上的拓展，其数据元素既可以是原子，也可以是广义表。类似的数据结构递归定义方法将在后续的树和二叉树结构部分进一步讨论。

字符串操作与一般线性表操作的不同是，常将字符串的部分或整体作为操作对象，而不是以单个字符元素为操作对象。

4.8 练习

1. 设二维数组 A[m][n],每个数组元素占用 k 个存储单元，第一个数组元素地址是 Loc(a[0][0]), 求按行优先顺序存放数组元素 a[i][j] (0≤i ≤ m-1, 0 ≤ j ≤ n-1)的存储地址为（）

A. Loc(a[0][0])+[(i-1)*n+j-1]*k　　B. Loc(a[0][0])+[i*n+j]*k
C. Loc(a[0][0])+[j*m+i]*k　　D. Loc(a[0][0])+[(j-1)*m+i-1]*k

2. 设二维数组 A[6][10],每个数组元素占 4 个存储单元，若按行优先顺序存放数组元素，a[0][0]的地址为 860，则 a[3][5]的地址为（）
A. 1000　　B. 860
C. 1140　　D. 1200

3. 若将 n 阶右上三角矩阵 A 按列优先顺序压缩存放在一维数组 B[0..$n(n+1)/2-1$], 第一个非零元素a_1,1 存于 B[0]中，则应存放到 B[k]中的非零元素$a_{i,j}$的下标i, j与k的对应关系是（）
A. i(i+1)/2 + j　　B. i(i-1)/2 + j-1
C. j(j+1)/2 + I　　D. j(j-1)/2 + i-1

4. 对 n 阶对称矩阵作压缩存储时，需要表长为（）的顺序表。
A. n/2　　B. n^2
C. n(n+1)/2　　D. n(n-1)/2

5. 对矩阵压缩存储是为了（ ）
A. 方便运算　　B. 方便存储
C. 提高运算速度　　D. 减少存储空间

6．广义表 LS=(((a, b, c)))的长度和深度分别是（）
A. 1 和 1　　B. 1 和 3
C. 1 和 2　　D. 2 和 3

7. tail(head(((a,b,c,d,e)))) =
A. a　　B. c,d
C. ( )　　D. (b, c, d, e)

8. 已知广义表 LS= ((a,b,c), (d,e,f)) , 运用 head 和 tail 函数取出 LS 中原子 e 的运算是（）
A. head(tail(LS))　　B. tail(head(LS))
C. head(tail(head(tail(LS))))　　D. head(tail(tail(head(LS))))

9. 下列说法不正确的是（）
A. 广义表的表尾总是一个广义表　　B. 广义表的表头总是一个广义表
C. 广义表难以用顺序存储结构　　D. 广义表可以是一个多层次的结构

10. 分析以逆对角线为界的左上三角矩阵和右下三角矩阵的压缩存储的地址计算方法。

11. 试将稀疏矩阵 A 的非零元素表示成三元组的顺序存储和十字链表存储的形式。

$$A=\begin{bmatrix}-7 & 0 & 0 & 10 & 15\\ 0 & 11 & 3 & 0 & 0\\ 0 & 0 & 0 & -5 & 0\\ 0 & 0 & 0 & 0 & 0\\ 0 & 0 & 14 & 0 & 0\end{bmatrix}$$

12. 试设计一算法，将十字链表表示的稀疏矩阵中的非零元素用顺序存储的三元组形式输出。

13. 设稀疏矩阵分别用三元组顺序和链式结构存储，设计算法实现 A+B 运算，并分析各自的时间复杂度。

14. 设有一个三对角矩阵（如图 4.4 所示），k 为压缩存储一维数组中某元素的序号，求该元素在矩阵中的下标 i 和 j。

15. 试设计一个稀疏矩阵非零元素表示的变换算法，将输入的三元组顺序存储变换成十字链表形式输出。

16. 若矩阵 A[m][n]中的某个元素 a_{ij} 是第 i 行的最小值，又是第 j 列的最大值，则称此元素为该矩阵的鞍点。设计一个算法，求矩阵的鞍点位置，并分析算法时间复杂度。

17. 编写函数 Replace(String &s， String t， String v)，将字符串 s 的所有子串 t 用字符串 v 替换。

18. 求下列广义表的运算结果。

(a) GetHead ((x, y, z))
(b) GetTail ((x, y, z))
(c) GetHead (((a, b),(x, y)))
(d) GetTail (((a, b),(x, y)))
(e) GetHead (GetTail (((a, b),(x, y))))
(f) GetTail (GetTail (((a, b),(x, y))))
(g) GetHead (GetTail (GetTail (((a, b),(x, y)))))

19. 设广义表 A=((a，b)，(), c，(d，(e, f)))，求表头、表尾，并画出两种存储结构示意图。

20. 词梯

[问题描述]

词梯：给定两个长度相等的单词，一个作为起点，一个作为终点。每次只改变一个字母，最终，作为起点的单词可以转变为终点的单词。

例如：code->cade->cate->date->data

词梯不是唯一的，如 work->play：

work fork form foam flam flay play
work pork perk peak pean plan play
work pork perk peak peat plat play
work pork perk pert peat plat play
work pork porn pirn pian plan play
work pork port pert peat plat play
work word wood pood plod ploy play
work worm form foam flam flay play
work worn porn pirn pian plan play
work wort bort boat blat plat play
work wort port pert peat plat play
work wort wert pert peat plat play

[基本要求]

（1）从 https://github.com/dwyl/english-words 中下载 words.txt 作为词汇表；

（2）给定两个长度相等的单词，一个作为起点，一个作为终点，寻找长度最短的词梯。

4.9 延伸阅读

字符串匹配中朴素模式匹配算法是最直观的解决问题的方法，虽然其最坏情况时间复杂度达到$O(nm)$，但在字符随机出现的情况下，最坏情况出现的概率极低，朴素模式匹配算法的期望运行时间甚至也可以达到线性。该算法常用于快速实现和简单匹配需求，在程序设计语言发展的初期有着广泛的应用。1971 年，库克定理[1]表明解决字符串匹配算法的时间复杂度最低为$O(n+m)$。

1977 年 Knuth、Morris、Pratt 三位学者提出 KMP 算法[2]，通过利用字符串特征向量来加快匹配的速度，使任意类型、不同特点字符串的模式匹配都能稳定地以$O(n+m)$的时间复杂度完成。该算法的提出使研究人员对字符串的模式匹配有了更高的观察理解视角，从而能够提出针对拓展应用的解决办法。紧接着，Boyer、Moore 提出更适用于大字符集模式、字符较为随机的应用场景下的 BM 算法[3]，利用坏字符规则和好后缀规则加快模式匹配的速度，成为较为常用的模式匹配方法。1987 年 Karp、Rabin 等人提出 KR 算法[4]，在模式匹配过程中引入散列方法，通过计算目标串中与模式串相同长度的子串的散列值，并与模式串的散列值进行比较来加快匹配速度。1990 年 Sunday 提出 Sunday 算法[5, 6]，利用目标串中待匹配子串的“下一位字符”来计算模式串需要右移的位数。KR 算法和 Sunday 算法因具有简单明了、实现简单的特点，在工程实践中得到广泛关注。

乔姆斯基谱系是计算机科学中刻画形式文法表达能力的一种文法体系，由语言学家 Noam Chomsky 于 1956 年提出[7]。该体系包括 4 个层次，其中 3 型文法（正规文法）生成正规语言，通常用来定义检索模式或程序设计语言中的词法结构，而这种文法规定的语言可以通过正则表达式获得。正则表达式是指定文本中匹配模式的字符串序列，为解决多模式匹配的问题提供了更为统一的表示方法，常用作字符串的匹配任务和替换任务。20 世纪 50 年代数学家 Kleene 开始形式化地描述正则语言[8]，描述了一种称为“正则集合”的数学符号，引入了正则表达式的概念。随后在 20 世纪 60 年代，UNIX 之父 Ken Thompson 提出了一种正则表达式搜索算法[9]，并基于该算法将正则表达式引入编辑器 qed 以及之后的编辑器 ed 中、然后又移植到了文本搜索工具 grep 中，使正则表达式逐渐成为 UNIX 系统中通用的字符处理方法。随着正则表达式的发展与完善[10]，多种语言加入对正则表达式的支持，使其成为字串处理的通用方法。

参考文献：

[1] COOK S A. The complexity of theorem-proving procedures[C]. in Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC’ 71). Shaker Heights: Association for Computing Machinery,1971:151-158.

[2] KNUTH D E, MORRIS J H, PRATT V R. Fast pattern matching in strings [J]. SIAM Journal on Computing, 1977, 6(2): 323-350.

[3] BOYER R S, MOORE S J. A fast string searching algorithm[J]. Communications of the ACM, 1977, 20(10): 762-772.

[4] KARP R M, RABIN M O. Efficient randomized pattern-matching algorithms[J]. IBM Journal of Research and Development, 1987, 31(2): 249-260.

[5] SUNDAY D M. A very fast substring search algorithm [J]. Communications of the ACM, 1990, 33(8): 132-142.
[6] GONNET G H, BAEZA-YATES R A. An analysis of the Karp-Rabin string matching algorithm[J]. Information Processing Letters, 1990, 34(5): 271-274.
[7] CHOMSKY N. Three models for the description of language [J]. IRE Transactions on Information Theory, 1956, 2(3): 113-124.
[8] KLEENE S C, Representation of events in nerve nets and finite automata[C]. In Conference: Automata Studies. Princeton: Princeton University Press, 1956, 34: 3-41.
[9] THOMPSON K, Programming techniques: regular expression search algorithm[J]. Communications of the ACM, 1968, 11(6): 419-422.
[10] AYCOCK J. A brief history of just-in-time[J], ACM Computing Surveys (CSUR), 2003, 35(2): 97-113.